Esame 11/02/25

1. (4) Definire intuitivamente e formalmente (mediante definizione semantica) cosa è un interprete. Descrivere e spiegare poi la struttura di un interprete (come pseudo-codice o come flow-chart).

Un interprete è un insieme di strutture dati e algoritmi per memorizzare ed eseguire programmi.
Semanticamente è una macchina astratta che esegue un linguaggio macchina e ritorna dei dati:

$$I^{L_0 L} : Prog^L \times Dati \to Dati$$

$$I^{L_0 L}(P^L, input) = output \quad t.c. \quad P^L(input) = output, \qquad P^L \in Prog^L, \; input \in Dati, \; output \in Dati$$

La macchina astratta è implementata in un linguaggio di livello inferiore L_0 ed esegue il linguaggio macchina L di livello superiore. La struttura dell'interprete è la seguente:

```
pc = 0
while (true) {
  istruzione = fetch(pc)
  op = decode(istruzione)
  args = null
  if op needs args {
    args = fetch_args(op)
  }

  switch (op) {
    case op1: exec(op1, args)
    case op2: exec(op2, args)
    ...
    case opn: exec(opn, args)
    case halt: return;
  }
  if opcode has result {
    store(result)
  }
  pc += 1
}
```

1. PC
2. While loop
3. Fetch
4. Decode
5. Get args if needed
6. Execute
7. Store result if present
8. PC++

2. (3) Si consideri l'insieme E definito induttivamente: $n \in E$, e se $e_1, e_2 \in E$ allora $e_1 + e_2 \in E$, $e_1 - e_2 \in E$. Si definiscano sugli elementi di tale insieme le seguenti funzioni: $v(n) = 1$, $v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2)$, e $o(n) = 0$, $o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1$. Si dimostri per induzione strutturale che $\forall e \in E.\, v(e) = o(e) + 1$.

$$E: \begin{cases} n \in E \\ e_1, e_2 \in E \Rightarrow e_1 + e_2 \in E \wedge e_1 - e_2 \in E \end{cases}$$

$$v: \begin{cases} v(n) = 1 \\ v(e_1 + e_2) = v(e_1 - e_2) = v(e_1) + v(e_2) \end{cases}$$

$$o: \begin{cases} o(n) = 0 \\ o(e_1 + e_2) = o(e_1 - e_2) = o(e_1) + o(e_2) + 1 \end{cases}$$

Dimostrare: $\forall e \in E.\; v(e) = o(e) + 1$

Dimostrazione per induzione strutturale

Caso base

$v(n) = o(n) + 1 \Rightarrow 1 = 0 + 1 \Rightarrow 1 = 1$ ✓

Passo induttivo

Dato un costruttore supponiamo che valga l'ipotesi indutiva sugli elementi dell'insieme e dimostriamo che il costruttore mantiene la proprietà

$e = e_1 + e_2$

Valuteremo e1+e2 senza perdita di generalità perchè e1-e2 si comporta in maniera analoga

Hp. Ind proprietà vale su $e_1 \wedge e_2$

$$v(e_1) = o(e_1) + 1$$

$$v(e_2) = o(e_2) + 1$$

Tesi: $v(e) = o(e) + 1$

$$v(e) = v(e_1 + e_2) = v(e_1) + v(e_2)$$

$$o(e) = o(e_1 + e_2) = o(e_1) + o(e_2) + 1$$

$$\overbrace{v(e_1) + v(e_2)}^{v(e)} = \overbrace{o(e_1) + o(e_2) + 1}^{o(e)} + 1$$

$\Downarrow$ i.i.

$$o(e_1) + 1 + o(e_2) + 1 = o(e_1) + o(e_2) + 1 + 1$$

$$o(e_1) + o(e_2) + 2 = o(e_1) + o(e_2) + 2$$

$$0 = 0 \checkmark$$

3. (5) Si consideri il programma sulla destra. Si dica cosa viene calcolato dall'assegnamento in caso di scoping statico (mostrando come vengono calcolati i link statici e come vengono risolti riferimenti non locali) e in caso di scoping dinamico (mostrando l'evoluzione della tabella centrale dei riferimenti (CRT) e come vengono risolti riferimenti non locali).

```
{int a=2;
void pippo(){
    a = a + 1;}
void pluto(){
    int a = 1;
    void minnie(int b){
        int a = 2*b;
        pippo();}
    pippo();
    minnie(3);}
a=3;
pluto(); }
```

```
{
  int a = 2;

  void pippo() {
    a = a + 1;
  }

  void pluto() {
    int a = 1;

    void minnie(int b) {
      int a = 2*b;
      pippo();
    }

    pippo();
    minnie(3);
  }

  a = 3;
  pluto();
}
```

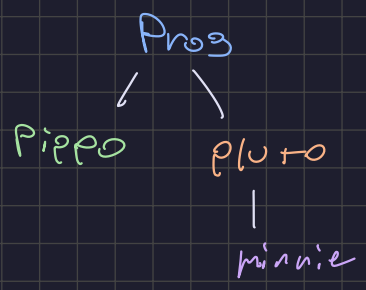
Statico

1 a = 2
2 a = 3
3 pluto() → a = 1
4 pippo() → a = a+1 → a = 4
5 minnie(3) → a = 2·b = 2·3 = 6
6 pippo() → a = a+1 → a = 5
7 END

↓

a = 5

Relazione di definizione



Dinamico

1 a = 2
2 a = 3
3 pluto()
4 int a = 1
5 pippo()
6 a = a+1 → a = 2
7 minnie(3)
8 int a = 2·b = 2·3 = 6
9 pippo()
10 a = a+1 → a = 7
11 END

↓

a = 3

Relazione di esecuzione

Prog → Pluto → Pippo → Minnie → Pippo

4. (6) Spiegare dove e perché nel programma è necessario parlare di regole di scoping, e dove (e perché) anche di regole di binding. Si dica quale risulta essere il valore finale di z nelle varie situazioni, ovvero con: (1) scoping statico; (2) scoping dinamico e deep-binding; (3) shallow-binding (e scoping dinamico).

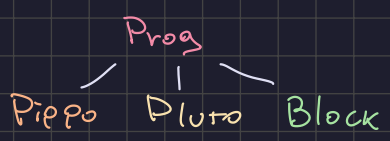
```
{int a = 4; int b = 3;
int pippo (int in){
    return a + b − in;
}
int pluto (int f(int b)){
    int b = 5; int a = 6;
    return f(b) + a;
}
{int b = 2;
int z = pluto(pippo); }
}
```

```
{
  int a = 4;
  int b = 3;

  int pippo(int in) {
    return a + b - in;
  }

  int pluto(int f(int b)) {
    int b = 5;
    int a = 6;
    return f(b) + a;
  }

  {
    int b = 2;
    int z = pluto(pippo);
  }
}
```

Si devono introdurre regole di scoping perchè le richiede la funzione pippo, siccome all'interno sono presenti variabili non locali e quindi bisogna risolvere i binding a cui fanno riferimento.

Si devono introdurre regole di binding perchè le richiede la funzione?

Ordine di definizione



Scoping statico

```
1 int a=4
2 int b=3
3 int b=2
4 int z= pluto(pippo)
5   int b=5
6   int a=6
7   return f(b)+a -> pippo(5)+6
8     return a+b-in -> 4+3-5=2 -> return 2+6=8
9 END
```

↓

z = 8

Ordine di esecuzione

Prog → Block → Pluto → Pippo → Pluto → Block → Prog

Scoping dinamico

```
1 int a=4
2 int b=3
3 int b=2
4 int z= pluto(pippo)
5   int b=5
6   int a=6
7   return f(b)+a -> pippo(5)+6
8     return 6+5-5
9   return 6+6
10 END
```

↓

z = 12

5. (5) Si dica per ognuno dei seguenti metodi di passaggio di parametri, quali sono i valori delle variabili dopo l'esecuzione del codice a destra: Passaggio per valore e Passaggio per riferimento. Dire quali sono i valori di x e y nel main al ritorno di pippo e descrivere dettagliatamente cosa accade nella memoria.

```
void pippo (int a, int b){
    a = 5 + a;
    b = 3b;
}
void main(){
    int x = 4, y=5;
    pippo(x,y);
}
```

6. (4) Siano $\rho = [x \mapsto 2, y \mapsto l_y]$ un ambiente dinamico con l_y locazione di tipo int, e $\sigma = [l_y \mapsto 3]$ una memoria. Descrivere le derivazioni della semantica dinamica per il comando $y := x + y$ nell'ambiente ρ a partire dalla memoria σ ($\rho \vdash \langle y := x + y, \sigma\rangle \to \cdots$).

$\rho = [x \mapsto 2,\ y \mapsto l_y]$ $\quad \Delta = [l_y \mapsto int\text{-}loc]$ $\quad \sigma = [l_y \mapsto 3]$

Suppongo che sia ben formato

$y := x+y$

La regola da applicare per eseguire questo comando è:

$$\frac{\rho \vdash \langle x+y, \sigma\rangle \to_e^* \langle k, \sigma\rangle}{\rho \vdash \langle y := x+y, \sigma\rangle \to_c \langle y := k, \sigma\rangle} \qquad (1)$$

Per ottenere k bisogna prima di tutto valutare l'espressione x+y, per farlo bisogna prima valutare x e y separatamente:

- Valutazione di x $\quad \rho \vdash \langle x, \sigma\rangle \to_e \langle 2, \sigma\rangle, \quad \rho[x] = 2$
- Valutazione di y $\quad \rho \vdash \langle y, \sigma\rangle \to_e^* \langle 3, \sigma\rangle, \quad \sigma[\rho[y]] = 3$

Ora posso valutare l'espressione intera

$$\frac{\rho \vdash \langle x, \sigma\rangle \to_e \langle 2, \sigma\rangle \qquad \rho \vdash \langle y, \sigma\rangle \to_e^* \langle 3, \sigma\rangle}{\rho \vdash \langle x+y, \sigma\rangle \to_e \langle 2+3, \sigma\rangle} \ \rho[x] = 2 \wedge \sigma[\rho[y]] = 3$$

Applico l'assioma per calcolare il valore finale dell'espressione

$$\rho \vdash \langle 2+3, \sigma\rangle \to_e \langle 5, \sigma\rangle$$

Adesso posso applicare la regola (1) per l'assegnamento

$$\frac{\rho \vdash \langle x+y, \sigma\rangle \to_e^* \langle 5, \sigma\rangle}{\rho \vdash \langle y := x+y, \sigma\rangle \to_c \langle y := 5, \sigma\rangle}$$

L'esecuzione di y := 5 ritorna una nuova memoria σ'

$$\rho \vdash \langle y := 5, \sigma\rangle \to_c \sigma[l_y \mapsto 5] = [l_y \mapsto 5] = \sigma'$$